Quantifying Process Model Generalization: A Generative N-gram Metric and a Cross-Paradigm Benchmark*

Christian Daniel Krengel and Tianhao Geng

Ludwig-Maximilians-Universität München, Munich, Germany

Abstract. Process discovery derives a process model from an event log, and the discovered model is conventionally judged along four quality dimensions: fitness, precision, simplicity, and generalization. Generalization, the degree to which a model accepts future valid behavior that is absent from the log, is the least validated of the four: published metrics follow incompatible paradigms and are almost never tested against a ground truth on real-life logs. We make four contributions. First, we argue for a strict construct definition: generalization covers only future valid behavior; penalizing invalid behavior is the task of precision. The trace model and the flower model operationalize the two poles and yield a purity litmus. Second, we present *ShadowGen*, a generative metric: a variable-order N-gram model of the log generates a synthetic shadow log of plausible-but-unseen traces, and a model is scored by replaying them under a graded and a strict reading. Third, we design a benchmark that validates ShadowGen and eight published metrics against a variant-based cross-validation ground truth on four agreement criteria. Fourth, we evaluate on five real-life logs that differ in scale, trace depth, and representativeness. ShadowGen tracks the ground truth on every log at a median of 3 seconds per model, faster than every published baseline. The widely used PM4Py metric correlates negatively with the ground truth on four of the five logs and fails the litmus on all five, and an adaptation of bootstrap generalization matches our calibration, corroborating the generative approach.

Keywords: Process mining · Process discovery · Generalization · Model quality · Conformance checking · Benchmark.

1 Introduction

Process discovery algorithms construct process models from event logs recorded by information systems [1]. A discovered model is conventionally assessed along four quality dimensions [9]: *fitness* (the model allows observed valid behavior), *precision* (it does not allow invalid behaviors), *simplicity* (it is structurally sparse) and *generalization* (it allows future valid behavior). An event log is only

* Report, M.Sc. Computer Science, LMU Munich.

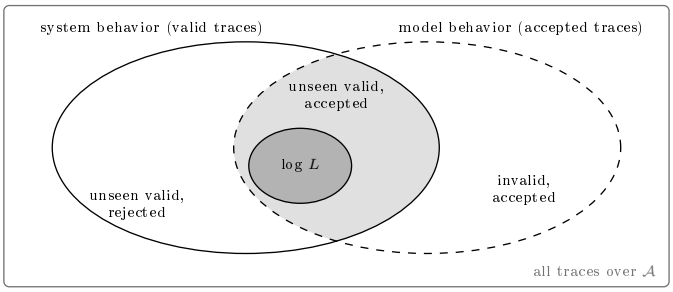

Fig. 1. Generalization as a set relation. The system produces the valid traces; the log is the observed sample; the model accepts some language of traces. Generalization asks how much of the unseen valid behavior the model accepts (the shaded region outside L) and is undermined by the lower-left region (future valid behavior the model rejects). The lower-right region, invalid but accepted behavior, is the concern of precision, not of generalization; replaying L itself is the concern of fitness. Every metric only ever sees L, so the two unseen regions must be estimated.

a sample of an underlying process: it contains the traces that were recorded, not all traces the process can produce. Generalization captures whether a model describes the process rather than memorizing the sample. Figure 1 shows the three sets involved: the behavior of the system, the behavior of the model, and the log that samples the former.

This report is about measuring generalization, not about discovering better models. Most published generalization metrics propose a measure and demonstrate it qualitatively; our stance is validation: a single-shot score is trustworthy only if it agrees with a direct, expensive measurement of generalization (hold-out acceptance). We also adopt a deliberately narrow definition, the acceptance of future valid behavior and nothing else, kept strictly separate from precision; this turns the otherwise-useless flower model into a diagnostic litmus test (Sect. 4.1). Finally, any metric and any hold-out ground truth is computed from a log, so its accuracy is bounded by how representatively the log samples the system [18]; we treat representativeness as a property of the data and span it deliberately across our datasets.

Measuring generalization is hard for a simple reason: it makes a claim about behavior that, by definition, is not in the data. The literature offers various metrics, but they approach the problem from fundamentally different angles: counting rarely used model parts [9], information-theoretic relevance of stochastic models [4], adversarial anti-alignments [13], generative adversarial networks [30], statistical bootstrapping [25], artificial negative events [8] and biodiversity-inspired species estimation [17]. These paradigms produce scores that are not clearly commensurable, and generalization metrics, unlike fitness and precision metrics, are known not to agree with one another [16]. No prior work validates them across paradigms against an empirical, log-derived generalization ground truth on common real-life logs. The practical consequence: no

validated consensus generalization metric exists. Even the generalization measure shipped in PM4Py [6], one of the most widely used process-mining libraries, does not track empirical generalization, as we show. A practitioner therefore has no validated, affordable instrument for the fourth quality dimension.

We pursue two goals. The first is a *new metric*. *ShadowGen*[1] learns a variable-order Markov model of the log: N-grams with Katz-style backoff [19]. From this, Good–Turing estimation [14] gives a per-context probability that the next event is new. ShadowGen then generates a synthetic *shadow log* of plausible-but-unseen traces and scores a model by how well it replays them [27]. The second goal is a *validated benchmark*. We test ShadowGen and eight published generalization metrics on eight discovery configurations and five real-life logs, and validate each metric against variant-based hold-out (both k-fold cross-validation and leave-one-variant-out), which measures acceptance of unseen-but-real behavior directly.

In sum, the contributions are: (i) a strict, precision-free construct definition with an operational purity litmus (Sect. 4.1); (ii) ShadowGen, an interpretable generative metric (Sects. 4–5); (iii) a cross-paradigm benchmark anchored by an empirical hold-out ground truth, with both construct poles included and a uniform compute budget (Sect. 6); and (iv) its results on five real-life logs. The headline result: ShadowGen tracks the cross-validation ground truth on every log (Pearson 0.986–0.999) at a median of 15 seconds per model, and it does so while scoring a model artifact in hand, which the ground truth itself cannot. The widely used PM4Py metric is anti-correlated with the ground truth on four of the five logs and fails the flower litmus on all five.

The remainder is structured as follows. Section 2 fixes notation and the concepts the method builds on. Section 3 surveys related metrics by paradigm and states the gap. Section 4 presents the construct, the metric, its pseudocode, a worked example, and a complexity analysis. Section 5 describes the implementation and where it deviates from the formal method. Section 6 reports the benchmark: setup, results, discussion, and threats. Section 7 concludes.

2 Background

This section fixes the notation and the concepts the report builds on: event logs, process models and their discovery, replay-based conformance, hold-out evaluation, and N-gram language models.

2.1 Event logs

Let $\mathcal{A}$ be a finite set of activities. A trace $\sigma = \langle a_1, \ldots, a_k \rangle \in \mathcal{A}^*$ is a finite sequence of activities; $|\sigma|$ is its length, and $\text{suffix}_n(\sigma)$ denotes the sequence of its last n activities. An event log L is a finite multiset of traces; $|L|$ is the number

[1] The implementation package keeps the legacy name `HybridGen`, from an earlier design that paired the generative score with a structural term. That term was retired (Sect. 4.2), so the metric here is the pure generative component.

of traces (cases) it contains. In general, for a multiset C we write $C[x]$ for the multiplicity of x and $|C| = \sum_x C[x]$ for the total count. Traces with an identical activity sequence form a *variant*; we write $V(L)$ for the set of variants and, for a variant $v \in V(L)$, $\#v$ for its case count. Real logs are typically dominated by a few frequent variants and a long tail of rare ones.

2.2 Process models and discovery

A process discovery algorithm (a miner, for short) maps an event log to a process model; in this report the model class is accepting Petri nets, and a model's language is the set of traces it accepts. Two extreme constructions bound the model space: a *trace model*, one isolated path per observed variant, accepting exactly the observed variants, and a *flower model*, all activities in a single self-loop, accepting every sequence over $\mathcal{A}$.

2.3 Token replay and its two readings

Token-based replay [27] replays a trace on a net, firing transitions and accounting for produced, consumed, missing, and remaining tokens. It yields two distinct quantities that this report keeps separate:

- a graded trace fitness in $[0, 1]$ derived from the token bookkeeping, which awards partial credit (a trace the net cannot actually execute end-to-end can still score, e.g., 0.7);
- a boolean perfect-replay flag: the trace replays with no missing or remaining tokens, i.e., it is in the model's language.

The fitness of a log is the mean trace fitness; the acceptance rate is the fraction of perfectly replayed traces. As Sect. 6.2.5 shows, the graded and boolean readings can diverge sharply, and the gap between them is itself informative.

2.4 Hold-out evaluation

The most direct operationalization of generalization is hold-out evaluation: discover a model on a subset of variants, then measure how well the withheld variants replay. Variant-based k-fold cross-validation and leave-one-variant-out realize this at increasing granularity, and cross-validation fitness has been used as a generalization measure when benchmarking discovery algorithms [5]. Hold-out is expensive (leave-one-variant-out requires one discovery per variant) and, importantly, is only defined when the model can be rediscovered from log subsets, so it evaluates a discovery algorithm rather than a model artifact in hand. These properties make it ideal as a validation reference but impractical as a routine metric.

Table 1. Published generalization paradigms and this report's metric. "Blended in" names what enters the score besides acceptance of unseen valid behavior, the construct of Sect. 4.1.

ID	Paradigm	Judges a model by	Unseen behavior probed	Blended in
M1	ShadowGen (this report)	replay of a generated shadow log of unseen traces	yes: recombinations and novel events	nothing (pure acceptance)
M2	structural counting [9,6]	how often model parts are visited when replaying the log	no	structure
M3	entropic relevance [4]	bits to encode the log under a stochastic model	no	fitness and precision
M4	anti-alignments [13]	the model's maximally deviating runs	model runs, not future traces	precision (paired measure)
M5	neural / GAN [30]	F-measure against GAN-sampled traces	yes: GAN samples	precision (F-measure)
M6	bootstrap [25]	recall against resampled, crossover-bred traces	partly: recombinations only	nothing (recall); the shipped tool reports an F-measure
M7	species estimation [17]	coverage profiles of original and simulated logs	no	diversity
M8	pattern-based [26]	alignment realization of mined behavioral patterns	no: patterns from the log	structure
M9	negative events [8]	replay against artificial negative events	no: negatives from the same log	precision-shaped signal

2.5 Variable-order language models

An N-gram language model predicts the next symbol of a sequence from its preceding context. Two classical techniques make such models robust on sparse data: *Katz backoff* [19] predicts from the longest recent context that has adequate statistical support, falling back to shorter contexts otherwise, and *Good–Turing estimation* [14] estimates the probability mass of unseen events from the number of singletons (events observed exactly once). Section 4.3 instantiates both for event logs.

3 Related Work

We group published generalization metrics by paradigm. Table 1 compares them along the two construct questions this report keeps asking: does the metric confront the model with unseen behavior, and what does it blend into the score besides acceptance of valid behavior? The identifiers M2–M9 name the metrics throughout the report; M1 is our own metric. Below we summarize each paradigm, review how such metrics have been evaluated so far, and state the gap.

3.1 The paradigms in brief

Structural counting (M2), the generalization metric built into PM4Py [9,6], deems a model general when its internal parts are visited often while replaying the log; it is trivial to compute but never confronts the model with unseen behavior. Alignment-based generalization [1], which weights how often each model

state is revisited during replay, shares this blind spot, so we group it with M2 and do not run it separately. Negative-event generalization (M9) completes the log with events assumed forbidden and scores the model against them [8]; the negatives are induced from the same log the model was mined from, which folds a precision-shaped signal into the score. Entropic relevance (M3) measures the bits needed to encode the log under a *stochastic* model [4], unifying fitness- and precision-like aspects in one quantity; it requires a stochastic model that a plain Petri net does not provide. A closely related line measures stochastic conformance rather than generalization: Earth Mover's Stochastic Conformance [22] scores the distance between the stochastic languages of the model and the log. Like entropic relevance it needs a stochastic model and blends fitness and precision into a single number, so we treat it, too, as outside a pure generalization comparison.

Anti-alignments (M4) derive precision and generalization from a model's maximally deviating runs [13]; the underlying search is exponential. AVATAR (M5) trains a sequence GAN to approximate the system and scores the harmonic mean of fitness and precision against sampled traces [30]; it is the closest paradigm to ours, but it uses a neural generator at hours of GPU training per log, and its harmonic mean folds precision in. Bootstrap generalization (M6) resamples the log and breeds recombined traces via genetic crossover [25]; its target, model–system recall, coincides with our construct, and it is the strongest baseline in our study, but its test traces are recombinations of observed material, so it injects no genuinely novel events.

SpeciAL (M7) transfers species-richness estimation from ecology to event data [17]: activities or N-grams are species, and coverage profiles quantify how representative a log or a simulated log is; we use the coverage ratio between model-simulated and original logs as its generalization proxy. Pattern-based generalization (M8) mines behavioral patterns from the log (tandem repeats, concurrency) and checks, via alignments, whether the model realizes them with generalizing structures [26].

3.2 Membership versus graded semantics

Textbook definitions are membership-shaped (the likelihood that previously unseen but valid behavior is allowed [1]; yes/no per trace), whereas token- and alignment-based fitness made the prevailing practice graded [27]. Our metric reports both readings of the shadow log and validates each against its own ground truth.

3.3 Evaluating the metrics themselves

A few studies assess the metrics rather than the models. Janssenswillen et al. [16] found that, unlike fitness and precision, generalization metrics *do not agree with one another*, and a follow-up on synthetic known-system logs judged objective measurement "nearly impossible" [15]. Others check conformance measures

against axioms [2,29] or benchmark discovery algorithms using these metrics as given [5].

3.4 The gap, and how ShadowGen differs

Three gaps recur. First, no cross-paradigm validation: metrics have been compared to one another [16] or on synthetic known-system data [15], never against an empirical hold-out ground truth on common real logs, so it is unknown which paradigm actually tracks held-out acceptance. Second, opacity: the most faithful baselines (neural, bootstrap) return a number with little insight into why a model scored as it did. Third, construct drift: several metrics fold precision in (Table 1, last column), which conflates two orthogonal failure modes. ShadowGen differs from every other row of the table in combining three properties: it is validated against a cross-validation ground truth and shown to track it; it is interpretable, exposing per-context novelty, mutation flags, an openness profile, and a strict acceptance reading; and it is construct-pure by design, with a benchmark that makes construct drift visible through the flower-model litmus.

4 Method

This section defines the construct ShadowGen measures (Sect. 4.1), the scoring framework (Sect. 4.2), the generator (Sect. 4.3), its pseudocode (Sect. 4.4), a worked example (Sect. 4.5), the complexity (Sect. 4.6), and the assumptions and limitations of the approach (Sect. 4.7).

4.1 Construct: what generalization is and is not

We adopt a strict construct: **generalization is the degree to which a model accepts future, valid behavior of the underlying process that is absent from the log, and nothing else.** Generalization is *not* precision. Detecting over-permissiveness (acceptance of *invalid* behavior) is the precision dimension's task. Folding a permissiveness penalty into a generalization score makes any mid-range value ambiguous: one cannot tell "rejects future valid behavior" from "accepts invalid behavior." That destroys the diagnostic value of the quality quadrant. Metrics that blend the two (for example, AVATAR's harmonic mean of fitness and precision, an F-measure) measure a legitimate but *different* construct and must not be read as pure generalization.

Two degenerate models operationalize the poles (Fig. 2). The *flower model* accepts every sequence over $\mathcal{A}$; its generalization is maximal *by definition*, and it is a useless model because of its precision and simplicity, not its generalization. A pure generalization metric must therefore score it ≈ 1. The *trace model* memorizes observed variants and accepts nothing unseen; it is the 0 pole. This yields a *construct-purity litmus*: a metric that scores the flower model below 1 demonstrably mixes precision or structure into its score. The metric is meant to

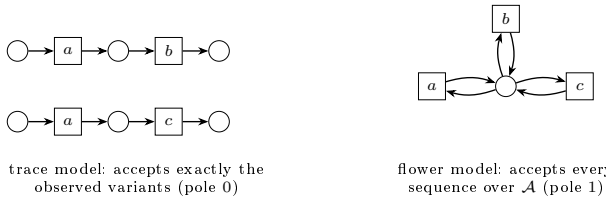

Fig. 2. The two construct poles for a log with variants $\langle a, b\rangle$ and $\langle a, c\rangle$ over $\mathcal{A} = \{a, b, c\}$. The trace model memorizes the variants and accepts nothing unseen; the flower model accepts everything, so a construct-pure metric must score it 1 (its uselessness is a precision and simplicity verdict, not a generalization verdict).

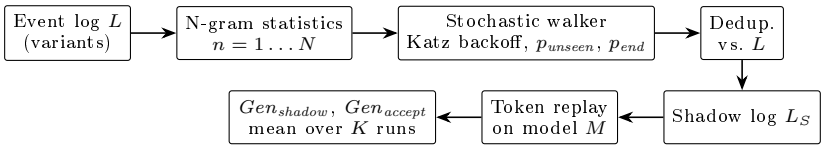

Fig. 3. ShadowGen pipeline. From the variants of the event log L, N-gram statistics up to order N feed a stochastic walker (Katz backoff; novelty probability p_{unseen}; termination probability p_{end}). Generated traces are deduplicated against L, so the shadow log L_S contains only unseen traces. The model M under evaluation enters only at the replay step, which yields the graded score Gen_{shadow} and the strict score Gen_{accept}, averaged over K repetitions.

be read *alongside* precision: a model with $Gen \approx 1$ and precision ≈ 0 is diagnosed as over-permissive by the precision column, exactly where that information belongs.

4.2 Framework

ShadowGen scores a model by the replay quality of a synthetic *shadow log*: traces that are plausible under the log's behavior but, by construction, absent from it. The generator is derived from the log alone, and the model under evaluation is consulted only at replay time (Fig. 3). This keeps the generation independent of the artifact being judged.

4.3 Shadow log generation

The generator is presented in walk order: the statistics it precomputes, the backoff that resolves a context, the novelty estimate, the mutation proposal, the exploitation step, termination, and the scoring of the finished shadow log.

4.3.1 Variant statistics. The log is compressed to its variants with case counts; every count below is weighted by how often the variant occurs ($\#v$). Let

$N \in \mathbb{N}$ be the context bound (an upper limit on the order, not a fixed operating point; its value, like every parameter value, is set in the evaluation protocol, Sect. 6.1). For each order $n \in \{1, \ldots, N\}$ and each observed context s (a length-n activity sequence), we record the successor multiset $C_n(s)$, where $C_n(s)[a]$ is the frequency with which activity a follows s, together with its support set $A_n(s) = \{a \in \mathcal{A} : C_n(s)[a] > 0\}$, and the termination counts: $T_n(s)$, the number of occurrences of s, and $E_n(s)$, the number of those occurrences at the end of a trace. Where the order is clear from the context length we drop the subscript and write $C(s)$, $A(s)$, $T(s)$, $E(s)$.

4.3.2 Katz backoff. At generation time the walker holds a partial trace σ and resolves its decision context to the deepest order with adequate support. Let $\tau \in \mathbb{N}$ be a support threshold. The *resolved order* R is the largest $n \leq N$ such that the suffix $s = \text{suffix}_n(\sigma)$ satisfies $|C_n(s)| \geq \tau$ (at least τ observed successor occurrences, Sect. 2) *and* the Good–Turing mass below is < 1. If no order qualifies, the walker falls back to n=1. N is therefore an upper bound: on sparse logs the effective order is reduced naturally.

4.3.3 Good–Turing novelty. For the resolved context s, the probability that the next activity is one never observed after s is estimated as

$$p_{unseen}(s) \;=\; \frac{N_1(s)}{|C(s)|}, \qquad N_1(s) = |\{a \in A(s) : C(s)[a] = 1\}|, \tag{1}$$

i.e., the number of singleton successors over the total successor count. With probability $p_{unseen}(s)$ the walker *mutates*; otherwise it *exploits*.

4.3.4 Mutation proposal (Katz-consistent). When a mutation fires, the inserted activity must be novel in the resolved context yet plausible. We draw it from the deepest *shorter* context that offers a successor unseen at the resolved order R. Formally, we take the largest order $m \in \{1, \ldots, \min(|\sigma|, N)\}$ whose candidate set

$$B_m \;=\; A_m(\text{suffix}_m(\sigma)) \setminus A_R(s) \tag{2}$$

is nonempty, and sample the inserted activity from B_m (weighted as in the exploitation step below). If no such m exists, we fall back to the global activity frequencies restricted to $\mathcal{A} \setminus A_R(s)$. A short observation shows that this search is a genuine backoff. Every occurrence of a context is also an occurrence of its shorter suffixes with the same next activity, so support sets can only grow as the context shortens: $A_n(\text{suffix}_n(\sigma)) \subseteq A_m(\text{suffix}_m(\sigma))$ for $m \leq n$. Orders $\geq R$ therefore never offer a candidate outside $A_R(s)$, and the first candidates appear at order $R{-}1$ or below, one level below the resolved order.

4.3.5 Exploitation: two sampling modes. For a non-mutation step the walker samples a seen successor $a \in A(s)$ with probability proportional to $w(c)$,

where $c = C(s)[a]$ is the observed successor count and $w : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is a weighting function. We support two weighting functions:

$$w_{\text{mle}}(c) = c \qquad \text{vs.} \qquad w_{\log}(c) = \ln(c+1). \tag{3}$$

The `mle` mode samples from the maximum-likelihood estimate of the next-activity distribution (proportional to observed frequency), so the shadow log mirrors the *expected* future. The `log` mode deliberately flattens the distribution, up-weighting rare paths; it is a rare-behavior *stress test*. The `mle` mode is the default and is what the evaluation reports; the calibration case for it is measured in Sect. 6.2.4. The same weighting governs the mutation-proposal sampling, but never the mutation *rate* of Eq. (1), which always uses raw counts.

4.3.6 Context-aware termination. Whether the walk ends after context s is decided by a termination probability resolved with the same backoff:

$$p_{end}(s) \;=\; \frac{E(s)}{T(s)}. \tag{4}$$

Conditioning termination on the n-gram context (rather than the last activity alone) fixes premature termination when activities that legitimately end traces appear early in a generated trace.

4.3.7 Deduplication and scoring. Each generated trace is compared against the set of original variants and regenerated if it is a copy (up to a retry cap), so the shadow log contains only unseen sequences. A shadow log L_S of $\theta \in \mathbb{N}$ traces (θ capped by $|L|$) is replayed on the model; generation and replay are repeated $K \in \mathbb{N}$ times and the scores averaged. We report two scores: Gen_{shadow} (mean graded trace fitness, the headline) and Gen_{accept} (fraction of perfectly replayed traces, the strict reading). We also report each score separately for shadow traces that contain a mutation and those that do not, which shows how a model handles genuinely novel events versus mere recombinations.

4.4 Algorithm

Algorithm 1 gives the generation procedure; statistics gathering and scoring are described above. The function rand() draws a fresh uniform random number from $[0, 1)$; all draws are seeded (Sect. 5.2).

4.5 Running example

Figure 4 works the method end to end on a toy log over $\mathcal{A} = \{a, \ldots, e\}$; for this example only, the context bound is N=2 and the threshold τ=3 (the benchmark values are fixed in Sect. 6.1). The four variants yield the N-gram statistics

Algorithm 1 Shadow-log generation (one iteration)

Require: event log L; context bound $N \in \mathbb{N}$; support threshold $\tau \in \mathbb{N}$; shadow-log size $\theta \in \mathbb{N}$; length cap $\lambda \in \mathbb{N}$; weighting function $w : \mathbb{N} \to \mathbb{R}_{\geq 0}$

build C_n, A_n, T_n, E_n for $n = 1..N$ from the variants of L (Sect. 4.3); collect the start-activity distribution and the variant set $V(L)$
$L_S \leftarrow \emptyset$
for $i = 1$ to θ **do**
  repeat
    $\sigma \leftarrow \langle$ sample start activity $\rangle$
    $mut \leftarrow$ false
    while $|\sigma| < \lambda$ **do**
      resolve context s and order R by Katz backoff with threshold τ
      if rand() $< p_{end}(s)$ **then break**
      end if
      if rand() $< p_{unseen}(s)$ **and** a novel candidate exists (Eq. (2)) **then**
        append a novel activity from B_m, weighted by w; $mut \leftarrow$ true
      else
        append a seen successor $a \in A(s)$, $\propto w(C(s)[a])$; **break** if dead end
      end if
    end while
  until $\sigma \notin V(L)$ **or** retry cap reached
  $L_S \leftarrow L_S \cup \{(\sigma, mut)\}$
end for
return L_S

excerpted in the middle panel. The first generated trace, $\langle a, c, e \rangle$, is a pure recombination: every step is attested (both c after a and e after c occur in the log), the sequence as a whole is unseen, deduplication confirms it is no variant, and it carries $mut =$ false. The second trace shows the mutation machinery. At $\sigma = \langle a, b \rangle$ the resolved order is R=2, since $|C(\langle a, b \rangle)| = 5 \geq \tau$, with successors $A_2(\langle a, b \rangle) = \{c, d\}$; the singleton d sets the novelty rate $p_{unseen} = 1/5$. When the draw fires, the proposal backs off to the deepest shorter context that offers something new (Eq. (2)): the order-1 context $\langle b \rangle$ offers $\{c, d, e\}$, so e (novel after $\langle a, b \rangle$, yet attested after $\langle b \rangle$) is inserted, and the trace is flagged $mut =$ true.

Figure 5 shows the same two-job split of backoff at realistic counts. An oversparse deep context is skipped entirely; the deepest supported order sets *how often* to invent an event (the rate); a still shorter one sets *what* to invent (the proposal). The inserted activity is novel in the exact context, yet attested in a related shorter one, and the resulting trace (absent from the log) is flagged as a mutated shadow trace.

4.6 Time and space complexity

Let $|V(L)|$ be the number of variants, $E_V = \sum_{v \in V(L)} |v|$ the summed length of the variants, $A = |\mathcal{A}|$ the alphabet size, N the context bound, θ the shadow-log size, and K the iteration count (all as above). Let $\ell_{\max} = \max_{v \in V(L)} |v|$ be the

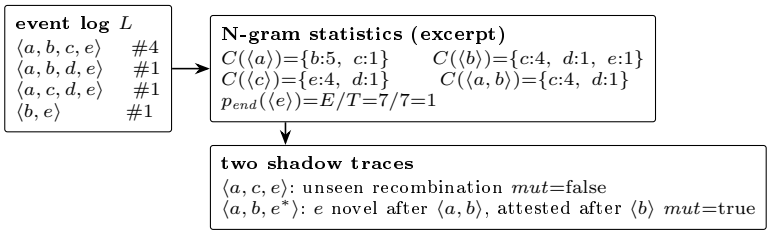

Fig. 4. Worked example (N=2, τ=3). A toy log of four variants, the successor multisets its N-grams induce, and two generated shadow traces: a recombination of attested steps, and a Katz-consistent mutation (e^*) whose rate came from the resolved order-2 context and whose proposal came from the order-1 context.

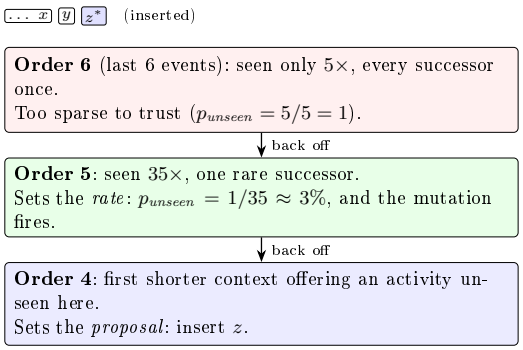

Fig. 5. Katz-consistent mutation at realistic counts (illustrative; activities abstracted to x, y, z). Backoff sets both *how often* to invent an event (the rate, resolved at order 5) and *what* to invent (the proposal, found at order 4): novel in the exact context, yet attested in a related shorter one.

length of the longest variant; the walk cap is set data-driven to $\lambda \approx 2\ell_{\max}$. Let $|M_P|$ be the size of the Petri net under evaluation, measured as its number of places, transitions, and arcs. Building the N-gram statistics is $O(N\,E_V)$ in time and $O(N\,G)$ in space for the G stored distinct n-grams. Generation is $O(\theta\,\lambda\,\bar{b})$ with mean branching factor $\bar{b} \le A$, since context resolution and termination are memoized on the last N activities, so each distinct context is resolved once. Token replay is roughly linear in trace length and net size [27], $O(\theta\,\lambda\,|M_P|)$. Per evaluated model the total is $O\big(K\,(N\,E_V + \theta\,\lambda\,\bar{b} + \theta\,\lambda\,|M_P|)\big)$.

One observation matters. The cost is linear in the log *through its variants*, not its cases: variant compression makes the metric scale with behavioral diversity rather than volume. This contrasts sharply with the leave-one-variant-out ground truth, which performs $|V(L)|$ full discoveries ($O(|V(L)| \cdot \text{discover})$) and is orders of magnitude more expensive on variant-rich logs.

4.7 Assumptions, scope, and limitations of the approach

ShadowGen assumes a control-flow view over a finite activity alphabet: traces are activity sequences, and other event attributes are not used. It assumes the log is a representative-enough sample that local N-gram statistics are meaningful; on extremely sparse logs the backoff collapses toward n=1, reducing the metric to a 1-gram directly-follows generator, which remains a reasonable if weaker baseline (measured as the ablation in Sect. 6.2.4). The shadow log models future behavior as recombinations of observed local patterns plus a calibrated tail of novel events; behavior driven by long-range or data-dependent dependencies beyond order N is not captured. The method has three parameters (the context bound N, the support threshold τ, the iteration count K), and a limitation is that they must be set upfront; per-dataset hyperparameter tuning is a possible refinement. Our protocol instead fixes them once for all logs (Sect. 6.1), which keeps the metric as parameter-free in use as fitness and precision are. Finally, ShadowGen measures only the generalization construct of Sect. 4.1 and is meant to be read alongside a precision metric, not in isolation.

5 Implementation

This section describes the reference implementation and the benchmark harness: the architecture (Sect. 5.1), the design decisions that matter for validity (Sect. 5.2), how each external metric was integrated (Sect. 5.3), and where the implementation deviates from the formal method (Sect. 5.4).

5.1 Architecture

The metric is implemented in Python using PM4Py [7] for log handling, discovery, and token replay. The code is organized as a registry of frozen modules: the released metric and its 1-gram ablation variant (Sect. 6.1) are separate modules loaded by name, so every number in this report regenerates from the exact code that produced it. The benchmark layer adds model preparation, per-method bridges (including subprocess bridges to Java/Docker baselines), the ground-truth scripts, and a runner with a uniform `evaluate_miner(log, miner)` entry point.

5.2 Key design decisions

Determinism. All random number generators are seeded (*seed*=42) so that every cell is reproducible; the residual nondeterminism of the replay library itself is measured in Sect. 6.4. **Provenance.** Every (log, miner, method) cell writes a sidecar JSON with exact parameters, raw per-iteration scores, runtime, and integrity counters (`duplicates_kept` and `truncated_traces`, which surface any deviation of the shadow log from its specification); these files are the single source of truth for all tables, and a result without a matching config is invalid.

5.3 Baseline integration realities

Integrating the eight external metrics (M2–M9, Table 1) surfaced engineering realities that materially affect the evaluation and must be reported as such. The PM4Py metric (M2) is a one-line library call. AVATAR (M5) runs in a TensorFlow 1 Docker image and required a greedy longest-match decoder to reconstruct multi-word activity names from the GAN's token output. SpeciAL (M7) is a Python package used directly; on L5 we use the released PyPI package (`special4pm` 0.1.3), on L1–L4 the source copy provided by the authors' repository, a version skew we disclose (Sect. 6.4). Anti-alignments (M4) and pattern-based generalization (M8) run only on the small logs and finish only part of the miners (Sect. 6.2.1). A third, entropic relevance (M3), needs a Petri-net-to-stochastic-automaton conversion, reported two ways. At scale we approximate it with a per-miner DFG simulation (PM4Py `play_out`, 5 000 traces) converted to a probability automaton via the open-source `relevance.jar` from the Entropia toolset [24] (JDFG2Aut + Relevance); this runs on all five datasets but takes its transition probabilities from the simulated DFG frequencies, not from the net. For the construct test we build the automaton properly, from each net's reachability graph with silent transitions collapsed and the result determinized over visible labels, probabilities from log replay, scored with Entropia's entropic relevance directly. The proper conversion is exact but bounded: on L1 two of the six non-degenerate miners have no finite automaton (unbounded or state-exploding reachability), and where it runs it fails the flower litmus (Sect. 6.2.1). Both are additionally reimplemented in plain Python and validated against the Entropia tool (3-decimal agreement on every L1 automaton).

Bootstrap (M6adapted) is a construct-faithful adaptation: its target, model–system recall [25], is exactly the construct we adopt (Sect. 3). Our bridge runs the genuine `bsgen` bootstrap-with-breeding *sampler* and scores the bred traces by PM4Py token-replay fitness, i.e. a *graded* model–system recall, the way Gen_{shadow} grades the shadow log. The sampler itself is our reimplementation of the paper's published algorithms (the original source is not public); it reproduces the published L1 values within 0.008 on every cell. We deliberately avoid the Entropia `-bgen` tool's harmonic mean of model–system precision and recall: that F-measure folds precision back in and is no longer the paper's recall-based generalization, and it fails the flower litmus of Sect. 4.1 (measured in Sect. 6.2.3). We carry it through the benchmark as its own method id (M6original) so the contamination stays measurable at scale (Sect. 6.2.4). The price of our choice is twofold. First, M6adapted as we report it is not the off-the-shelf Entropia `-bgen` output but our own token-replay scoring of its genuine `bsgen` sampler, so it is an adaptation, not a drop-in published baseline. Second, that scoring is the conformance core of our metric and of R1, a shared-ruler effect we flag in Sect. 6.4.

5.4 Deviations from the formal method

Three deviations are worth recording. (i) The N-gram model is rebuilt each of the K iterations rather than once; this is a performance artifact, not a seman-

tic one, and the complexity of Sect. 4.6 already carries the factor K. (ii) The strict reading counts a shadow trace as accepted when PM4Py's token replay flags it as perfectly fitting (`trace_is_fit`). That flag is a fast but approximate test of language membership, which is properly decided by a cost-zero alignment. On nets with invisible (silent) or duplicate-label transitions the two can disagree, so our acceptance number is a proxy; Sect. 6.2.5 quantifies the disagreement per net class. (iii) The data-driven length cap never binds on L1/L2 (`truncated_traces`=0). On the deep-trace logs it does, and it pays: on L4 the cap alone improves calibration from MAE 0.038 to 0.027 (log weighting throughout, cap the only change; the headline MLE configuration reaches 0.016).

6 Evaluation

We measure how well each metric agrees with an empirical generalization ground truth across discovery configurations and logs, and we attribute the proposed metric's behavior to specific design decisions through a measured ablation. Section 6.1 fixes the setup, Sect. 6.2 reports the results, Sect. 6.3 discusses them, and Sect. 6.4 states the threats to validity.

6.1 Experimental setup

The setup has four ingredients: the logs, the discovery configurations, the methods with their reference measurements, and the protocol with its compute budget.

6.1.1 Datasets. From a profiled catalog of 21 public logs we selected five (Table 2), named L1–L5, spanning trace depth, variant diversity, scale, and, most deliberately, representativeness. We quantify the latter by the trace-based log representativeness approximation (TLRA, defined in Table 2) [18]: the empirical probability that a new case repeats a known variant, and the axis shown to impact the accuracy of generalization estimation. TLRA ranges 0.19–0.95 across the selection. The same evaluation battery runs on every log. We present L1 (Sepsis) in full as the primary case, carrying the cross-paradigm comparison and its figures; L2 (BPI 2013 Incidents) corroborates the litmus and the acceptance reading; L3–L5 extend the study to logs of up to 2.5 million events (Sect. 6.2.4).

6.1.2 Discovery configurations. Eight discovery configurations per log span the construct's poles: six process discovery algorithms, namely Alpha and Alpha+ [3], Heuristics default and strict [31], and Inductive strict and infrequent [20,21], framed by the two degenerate poles of Sect. 4.1: a filtered trace model (top-50 variants by frequency, since the full trace model is intractable to replay on variant-rich logs) and a flower model. Models are discovered once per combination of log and configuration (a *cell*) and shared by all methods. We refer to the six algorithmic configurations as the six non-degenerate miners.

Table 2. Benchmark event logs. TLRA $= 1 - |V(L)|/|L|$ is the trace-based log representativeness approximation [18].

ID	Log	Cases	Events	Variants	Avg. len	TLRA
L1	Sepsis [23]	1,050	15,214	846	14.5	0.19
L2	BPI 2013 Incidents [28]	7,554	65,533	1,511	8.7	0.80
L3	BPI 2017 [10]	31,509	1,202,267	15,930	38.2	0.49
L4	BPI 2018 [12]	43,809	2,514,266	28,457	57.4	0.35
L5	BPI 2019 [11]	251,734	1,595,923	11,973	6.3	0.95

6.1.3 Methods and ground truth. Our metric is M1: ShadowGen as specified in Sect. 4. A single ablation variant replaces the variable-order model by its 1-gram floor, a directly-follows generator with the same Good–Turing novelty and termination machinery, and isolates what deep context contributes (Sect. 6.2.4). The external metrics are M2–M9 (Sect. 3, Table 1). Three reference measurements complete the battery. R1 is the empirical ground truth: variant-based 5-fold cross-validation fitness (3 shuffles), i.e. discover on four fifths of the variants, replay the held-out fifth. R2 is its finer-grained counterpart, leave-one-variant-out, with one discovery per held-out variant; it checks that the verdicts do not depend on the fold granularity. R3 is a floor, not a ground truth: the replay of uniformly random traces, which any metric claiming to measure more than permissiveness must beat. An acceptance-based variant, R1-accept, measures the fraction of held-out traces replayed perfectly and validates the acceptance reading.

6.1.4 Protocol and agreement criteria. ShadowGen runs at the defaults fixed here, once, for all logs: context bound N=6, support threshold τ=5, K=5 iterations of θ=1,000 shadow traces (θ capped by $|L|$), `mle` weighting, seed 42; each cell reports mean $\pm$ std. We benchmark at K=5 so every cell carries an error bar, but the shipped default is a single draw (K=1), which is five times faster and reproduces the result (Sect. 6.4). Each external method runs with its own stochasticity protocol. Agreement with the ground truth is reported on *four criteria*: Pearson (calibration shape), Spearman and Kendall's τ_b (ordering), mean absolute error (MAE, absolute calibration), and discriminative spread (max$-$min over the six non-degenerate miners). All four are computed over those six miners; the two poles are excluded and reported separately as litmus values. Rank correlation alone is too weak a test: as Sect. 6.2.2 shows, a random-replay baseline also reaches Spearman and Kendall 1.0. The context bound was fixed at N=6 via a mutation-rate sweep on L3 (the realized mutation rate peaks at N=6 and declines beyond, as backoff increasingly rejects sparse contexts; Fig. 6), and N=6 is held fixed across all logs so the metric is never tuned per dataset.

Compute budget. Every method is granted 1 hour of metric time per cell. Model discovery is excluded: models are discovered once per cell, cached, and

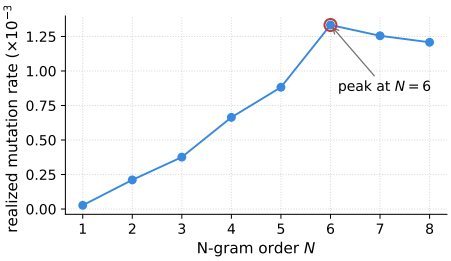

Fig. 6. Realized mutation rate vs. N-gram order on L3 (BPI 2017). The rate peaks at N=6 and declines beyond, as Katz backoff increasingly rejects over-sparse high-order contexts; N=6 is then held fixed on every log.

shared by all methods. The references (R1, R2, R1-accept) are exempt; they define the ground truth. A cell that does not return within the budget is recorded as a sentinel with the measured evidence, excluded from the agreement statistics, and not re-run. Since the working metrics complete a cell in seconds, this turns runtime into a first-class outcome: a method's inability to finish is a result, not an omission.

6.2 Results

We report the results in a fixed order: which methods return scores at all under the budget (Sect. 6.2.1); the cross-paradigm agreement on the primary log (Sect. 6.2.2); a controlled study of what contaminates a generalization score (Sect. 6.2.3); the agreement at scale, including the ablation (Sect. 6.2.4); the strict acceptance reading (Sect. 6.2.5); a direct test of the generator premise (Sect. 6.2.6); and runtime (Sect. 6.2.7).

6.2.1 Feasibility. Before any agreement can be compared, each method must return scores within the budget; this subsection reports which methods do, and documents that some cannot be applied to the larger logs in reasonable time. Under the compute budget, two published metrics (M4, M8) return on too few models to be scored: each finishes at most two of the six non-degenerate miners on any log, on the small logs only, and neither returns a non-degenerate miner at scale. Anti-alignments (M4) run the published implementation [13] under the same budget. It scores three of the eight L1 models and three of the eight L2 models, never more than two of the six non-degenerate miners, nothing on L3 or L4, and only the trace pole on L5; on L4 the 2.5 million event log does not finish even the alignment step within the hour. On L1 Inductive-infrequent aborts inside the solver on an internal index overflow, Alpha+ reaches 1% of its traces within the hour, Heuristics reaches 87% at the kill (the one near-miss) and Inductive-strict 0.1%, and the flower model never returns. Where M4 does score, the trace model reads 0.0, the correct pole. Pattern-based generalization (M8) [26] fails similarly:

it scores three of the eight L2 models in seconds (Heuristics 0.99, Heuristics-strict 0.999, and the trace model), crashes with a null alignment on the other five, and returns nothing on L1, L3, L4, or L5. Unlike M4, it exposes no progress counter, so its non-returns are wall-clock kills without a measured progress percentage. Its 0.90 on the trace model is the wrong pole, a memorizing model read as general. Neither returns all six non-degenerate miners on any log, so neither can be placed on the four-criteria agreement. Entropic relevance (M3) is feasible on all five logs as a directly-follows approximation (Sect. 5.3), but its agreement with the ground truth is unstable (per-log Pearson against R1 from -0.80 to $+0.95$), so we do not read it as tracking held-out fitness. Computed properly, from a stochastic automaton built from each net's reachability graph, it confirms the problem directly: on L1 two of the six non-degenerate miners (Alpha, Heuristics) have no finite automaton, and on the four that convert it places the flower model mid-field (52 bits against 22 to 62 for the real models), failing the flower litmus of Sect. 4.1. Extended to all five logs, the proper conversion builds on 24 of 40 cells (all 8 on the 4-activity L2, 3 of 8 on L5, at a 150k-marking cap); the deeper the concurrency, the fewer nets convert. Entropic relevance scores a fitness and precision blend, not generalization, so we keep it out of the agreement tables.

Negative-event generalization (M9) [8], run through the CoBeFra implementation in its shipped configuration (weighted artificial negatives with token replay), does not track held-out generalization on any log. On L1 it is anti-correlated with the ground truth over the six non-degenerate miners (Pearson -0.55, more negative than PM4Py), rating the well-generalizing Heuristics models near zero; on the four-activity L2 and the short-trace L5 it collapses, scoring every model 1.0 including the memorizing trace pole, so it discriminates nothing (MAE 0.18 and 0.23). It passes the flower litmus on the small logs (1.0) but mis-anchors the trace pole (0.48 on L1, 1.0 on L2 and L5), and it does not scale: on L3 only two of the six non-degenerate miners return within the hour and on L4 (mean trace length 57) none do. Its scores are also non-deterministic on nets with silent transitions (L1 Inductive-infrequent, std 0.07 across runs), the same token-replay tie-breaking that affects the acceptance proxy (Sect. 6.2.5). Because it is degenerate on L2 and L5 and infeasible at scale on L3 and L4, no scorable four-criteria result exists; like structural counting and entropic relevance it measures a fitness and precision blend rather than generalization, so we keep it out of the four-criteria agreement tables; its speed and (poor) accuracy still place it on the runtime pareto (Fig. 10). The run and driver are in `benchmark/m9/`.

AVATAR (M5) meets the one-hour budget on no log and is therefore a budget sentinel on all five; its two small-log scorecards, obtained off-protocol (two of the 3–5 published runs), are read for the flower litmus and the off-protocol calibration illustration (Figs. 7, 10; Sect. 6.2.2), never as agreement rows. The evidence is measured: a reduced L1 anchor (3 of 100 pre-epochs, 100 of 5,000 adversarial steps) already needs 88 minutes on the smallest log, its per-step rate (45 s) projects the full published configuration to $\sim$3–4 days of wall time (over 400 CPU-hours) per log on benchmark-class CPU hardware, L3 spent its full hour without leaving GAN pretraining, and L4 was killed at both a 16 and a 24 GB

Table 3. Agreement with R1 on L1 (six non-degenerate miners) and the flower litmus. τ_b is Kendall's tau-b. *M6adapted uses adapted scoring (Sect. 5.3).

Method	Pearson	Spearman	τ_b	MAE	Spread	Flower
M1 ShadowGen (ours)	0.996	1.00	1.00	0.023	0.71	1.00
M2 PM4Py	−0.35	−0.43	−0.20	0.17	0.09	0.91
M6adapted*	0.998	1.00	1.00	0.018	0.74	1.00
M7 SpeciAL	0.12	−0.46	−0.41	0.21	0.26	0.80
R3 Random	0.83	1.00	1.00	0.28	0.49	1.00

memory cap. The authors themselves report hours of GPU training per log [30], so on either hardware class AVATAR sits orders of magnitude beyond the one-hour budget. Its published configuration also caps sequences at 20 events, below the mean trace length of L3 (38.2) and L4 (57.4), so its scores there would truncate the behavior generalization asks about; raising the cap is a configuration change that only adds to the already prohibitive cost.

Two boundary results at scale: the published Entropia `-bgen` tool needs 6 to 10 minutes per L4 cell (five of eight cells return within the hour; the other three fail inside the tool), and SpeciAL scores six of eight L5 cells (Alpha and Alpha+ exceed the hour). In sum, M2, M3, M6adapted, M6original, and M7 score at scale with these sentinels; M5 exceeds the budget on every log; M4 and M8 return only on the small logs, always leaving models unscored, and no non-degenerate miner at scale. We also disclose five L4 cells kept in the tables despite overrunning the hour before returning, against the sentinel rule: M2 Inductive-strict (3.5 h), the 1-gram ablation's Inductive-strict (4.0 h), SpeciAL Heuristics-strict (2.2 h) and Heuristics (1.2 h), and the R3 floor's Inductive-strict (1.2 h). They are real measurements, and sentineling them would only remove scored cells from baselines that already fail, so we keep and flag them.

6.2.2 Cross-paradigm agreement on L1. Table 3 compares the paradigms against the cross-validation reference R1 on L1. Four findings stand out.

1. Generative and resampling paradigms track held-out fitness; structural and diversity paradigms do not. The most widely available metric (M2) correlates negatively with the ground truth, compresses six structurally very different models into a spread of 0.09, and ranks the two Alpha variants, the ground truth's bottom pair, above every other model. This is direct evidence that visit-frequency counting does not measure held-out generalization.
2. Rank correlation alone is a low bar. The random-replay floor R3 attains Spearman and τ_b of 1.0, because the Sepsis miners differ so strongly in permissiveness that almost any test orders them correctly. Validation must rest on the full set of criteria, which is where ShadowGen and bootstrap (MAE 0.023/0.018, spreads 0.71/0.74) separate from R3 (MAE 0.28, spread 0.49).
3. The flower litmus sorts the field. Pure-acceptance metrics (ShadowGen, M6adapted, R1–R3) score the flower model 1.0, as the construct requires;

M2, M7, and especially AVATAR (off-protocol flower 0.40; Sect. 6.2.1) reveal precision/structure contamination. That M6adapted lands in the first group is independent corroboration, not coincidence: the bootstrap metric is itself defined as model–system recall [25], so a separate, published generalization measure also rates the flower model maximally general. AVATAR's low flower score, by contrast, follows from its construct (a harmonic mean of fitness and precision [30], i.e. an F-measure), which also explains why it scatters off the calibration line (Fig. 7).

4. Bootstrap is the closest competitor, with a caveat. M6adapted matches our calibration, but it scores with token-replay fitness, the same conformance engine as R1 and our own metric; part of its tight agreement is a shared-ruler effect rather than independent corroboration (Sect. 6.4).

The agreement is robust to the choice of cross-validation reference: against the finer-grained leave-one-variant-out (R2, 50/846 variants sampled), ShadowGen's ranking is identical (Kendall τ_b=1.0) and its calibration is unchanged on L1 (MAE 0.014); on L2 the ranking again holds while calibration stays strong (Pearson 0.97, MAE 0.048). The verdicts on every baseline are the same under R1 and R2. The agreement is robust to the miner sample too: extending L1 to eleven discovery configurations (the six non-degenerate miners plus five threshold variants) holds the calibration at Pearson 0.997, with a bootstrap 95% confidence interval of $[0.953, 0.999]$ (MAE 0.015, interval $[0.010, 0.021]$), so it is not an artifact of a six-point sample.

The litmus failures also replicate on L2: PM4Py scores the flower model 0.88, AVATAR 0.74 (off-protocol), and SpeciAL 0.94, all below the 1.0 the construct requires, while ShadowGen and the references stay at 1.0. Figure 7 plots each metric against the cross-validation ground truth over the six non-degenerate miners of all five logs; the L1 four-criteria detail is Table 3 and the per-cell scores are in the appendix (Table 8). The generative and resampling metrics stay on the diagonal on every log, while the structural and diversity metrics do not, which previews the scale result of Sect. 6.2.4 in visual form.

6.2.3 What contaminates a generalization score: a controlled bootstrap study. Bootstrap generalization is published as a generalization metric in its own right [25], so it is worth asking why the number a user typically reads off it (the Entropia `-bgen` F1 of model–system precision and recall) fails the flower litmus of Sect. 4.1. Because one bred `bsgen` sample can be scored several ways, we hold the sampler fixed and vary only the scoring (Table 9, appendix).

The two readings that include precision fail the litmus (flower 0.18, 0.10) and miscalibrate badly (MAE 0.59, 0.68); the two recall-only readings pass and track R1. The F1 still reaches Pearson 0.87, yet its MAE 0.59 and flower 0.18 expose it, which is why the protocol uses four criteria and not one. So it is the precision component, not the bootstrap sampler, that throws the bootstrap off, and this is why we report it (M6adapted) by token-replay fitness, faithful to the paper's recall construct (Sect. 5.3). This is the construct argument of Sect. 4.1 made

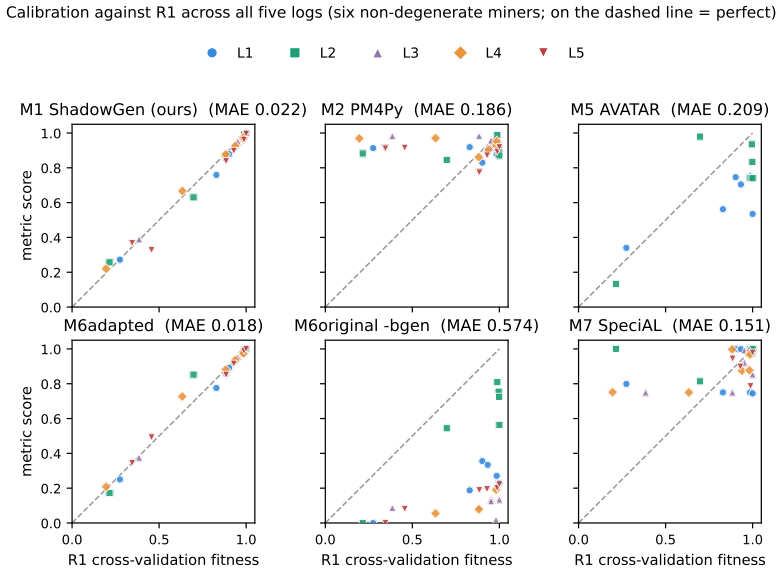

Fig. 7. Per-method calibration against the cross-validation ground truth R1 across all five logs (six non-degenerate miners per log, up to 30 points, one marker per log; dashed line $y{=}x$). The generative and resampling metrics (M1 ShadowGen, M6adapted) sit on the line on every log; the published Entropia `-bgen` F-measure of the same bootstrap sample (M6original) sits far below it (its F1 folds precision in, Sect. 6.2.3); M2 PM4Py forms a flat band that ignores the ground-truth gradient; M5 AVATAR (shown off-protocol on L1/L2 only, Sect. 6.2.1) and M7 SpeciAL scatter off the line. Metrics that could not run on every log appear with fewer points: AVATAR's sparse panel is itself the feasibility verdict.

empirical: folding precision into a generalization score is not merely conceptually wrong, it measurably destroys calibration and the litmus.

6.2.4 Agreement at scale (L3–L5) and the ablation. The three large logs multiply the event volume by up to 165× over L1 and change the regime: deeper traces (L4 averages 57.4 events), larger variant spaces (up to 28,457), and both extremes of representativeness (TLRA 0.35–0.95). Table 4 reports the agreement with R1 on all five logs; the per-metric scatter across those logs is Fig. 7, where ShadowGen's 30 non-degenerate cells sit on the diagonal from L1 through the 2.5 million event L4.

Six findings.

1. *The calibration holds at scale.* ShadowGen tracks R1 on every log: Pearson 0.986–0.999, MAE 0.006–0.043, spread 0.61–0.76, flower exactly 1.0 everywhere. The ranking is perfect on L1, L2, and L4; on L3 and L5 exactly one adjacent pair swaps (Spearman 0.94). Both swaps are benign: on L3 the pair

Table 4. Agreement with R1 on all five logs (six non-degenerate miners each): Pearson / MAE per log, and the flower-litmus range. AVATAR is a budget sentinel on all five logs (Sect. 6.2.1); the SpeciAL L5 entry covers the four miners that returned within budget; the Entropia `-bgen` L4 entry covers the three that returned; M4/M8 return too few cells to rank, and M9 is degenerate on L2/L5 and infeasible at scale, so neither is ranked here (its speed/accuracy point is in Fig. 10).

	L1		L2		L3		L4		L5		Flower
Method	Pear.	MAE	Pear.	MAE	Pear.	MAE	Pear.	MAE	Pear.	MAE	
M1 ShadowGen (ours)	.996	.023	.994	.019	.999	.006	.999	.016	.986	.043	1.00 (all)
M6adapted	.998	.018	.978	.034	.999	.006	.993	.020	.998	.014	1.00 (all)
M2 PM4Py	−.35	.172	.41	.184	−.65	.135	−.53	.208	−.33	.229	.88–.98
M7 SpeciAL	.12	.209	.12	.158	.63	.122	.76	.162	−.24	.079	.76–.94
M6original (`-bgen` F1)	.87	.591	.95	.250	.13	.760	.82	.724	.98	.619	.05–.53
R3 Random floor	.83	.281	.97	.116	.70	.265	.88	.176	.84	.196	1.00 (all)

is separated by 0.0004 in R1 (a tie for practical purposes), on L5 it is the two Alpha variants, both scored far below the field by metric and ground truth alike.

2. *A second generator lands on the same calibration.* The bootstrap adaptation (M6adapted) co-leads on every log (MAE 0.006–0.034, flower 1.0). We read this as corroboration rather than competition: two independently designed synthetic-log generators (bootstrap resampling with crossover breeding; a variable-order N-gram model with calibrated novelty), scored by the same acceptance reading, agree with the ground truth and with each other. The generative paradigm, not one specific generator, carries the result. The practical difference is cost (median 36.9 s against our 14.9 s per model) and the principled novelty injection that resampling lacks (Sect. 3).
3. *The precision contamination of the published reading replicates everywhere.* The Entropia `-bgen` F1 of the same sampler fails the litmus on every log (flower 0.05–0.53) and miscalibrates by an order of magnitude more than the recall-faithful adaptation (MAE 0.250–0.760). The L1 diagnosis of Sect. 6.2.3 is not an L1 artifact.
4. *PM4Py fails on every log.* Its Pearson is negative on L1, L3, L4, and L5 and 0.41 on L2, its spread never exceeds 0.14, and it fails the flower litmus on all five (0.88–0.98). SpeciAL shows no stable relationship either (Pearson −0.24 to 0.76) and rates the memorizing trace model at 1.0 on every log, the opposite pole error.
5. *Rank correlation stays a low bar at scale.* The random-replay floor R3 orders the miners nearly perfectly on every log (Spearman 0.94–1.0, τ_b 0.87–1.0) while miscalibrating by MAE 0.116–0.281. The four-criteria protocol is what separates the calibrated metrics from the floor, on every dataset.
6. *Deep context earns its keep on deep logs.* The 1-gram ablation is competitive on the two short-trace, high-TLRA logs (L2 MAE 0.036; on L5 it is even better calibrated, 0.026 against 0.043) but falls behind by an order of magnitude on L3 (MAE 0.066 against 0.006), by 2.3× on L4, and by 3× on L1. This confirms the prediction registered in the two-log study: the

variable-order model pays off exactly where decisions depend on deep context, and the honest converse is that on shallow repetitive logs a directly-follows generator suffices. The weighting choice behaves the same way: the default MLE weighting calibrates better than the log-weighted stress mode on L1–L4 (MAE 0.023/0.019/0.006/0.016 against 0.068/0.045/0.046/0.027), while on L5, the same shallow and repetitive regime, the flattened mode is slightly better calibrated (0.035 against 0.043).

One registered prediction is refuted. Following [18] we predicted that calibration error grows as TLRA falls. It does not: the lowest MAE occurs at TLRA 0.49 (L3, 0.006) and the highest at TLRA 0.95 (L5, 0.043), with no monotone trend across the five logs. At these scales, trace depth and alphabet size govern the difficulty of generalization estimation at least as much as trace-based representativeness does.

6.2.5 The acceptance reading. Gen_{accept} reports the fraction of shadow traces replayed perfectly, the strict membership-shaped reading. It is not a standalone quality score: on real logs it is nearly bimodal (most models strictly accept almost nothing unseen, the flower model accepts everything), so it carries little rank information. Its job is to support the construct by exposing a replay mechanic: token replay awards partial credit, so the memorizing trace model still reaches a graded 0.57 on L1 and Alpha+ 0.76 while accepting nothing unseen (Fig. 8); the strict reading puts them at exactly 0 and the flower model at exactly 1, the clean construct poles that graded fitness cannot provide. The inflation is tolerable, the trace model is a degenerate extreme in any case, but the strict reading is what makes it visible. Near-zero acceptances are true model properties, not metric failures: the acceptance ground truth (R1-accept, the fraction of held-out real traces replayed perfectly) agrees (on L1, Alpha+ 0.000, Heuristics 0.028; only the Inductive family genuinely accepts novel behavior). Validated against R1-accept, Gen_{accept} reaches Pearson 0.992–0.999 and MAE 0.021–0.076 on L1, L2, L3, and L5, and the poles anchor on every log including L4 (per-miner and per-log detail in Tables 6 and 7, appendix).

Two caveats bound the reading. Strict acceptance saturates at trace depth: on L4 (mean trace length 57.4) even held-out real traces barely replay perfectly (the ground truth's largest value is 0.30), since one deviation anywhere in a 57-event trace breaks a perfect replay; Pearson is therefore uninformative on L4 (MAE 0.057), while the poles still anchor exactly. And the membership test is a replay proxy (Sect. 5.4): measured against cost-zero alignments on L1 shadow traces, the replay flag agrees exactly on the Inductive-strict, Flower, and Heuristics nets and reaches 0.804 on Inductive-infrequent, all 196 disagreements one-directional over-accepts, so mid-range acceptance values on that net class are upper bounds.

6.2.6 The generator premise, tested directly. The metric's central premise is that the shadow log resembles valid future behavior. Section 4.2 argues this constructively; here it is measured. For every R1 fold partition

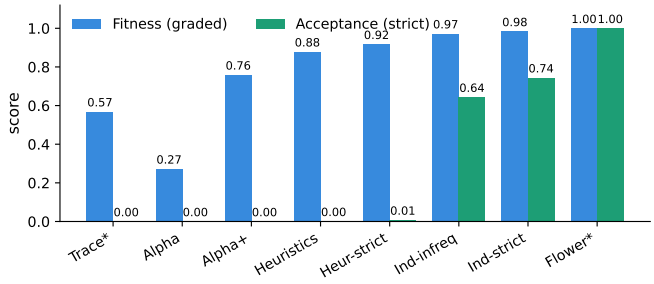

Fig. 8. Two readings of the shadow log on L1 Sepsis: graded fitness (partial credit) vs. strict acceptance (perfect-replay fraction). Acceptance gives the construct's clean poles (Trace 0.00, Flower 1.00), and the gap between the bars exposes partial-credit inflation: e.g. Alpha+ scores 0.76 fitness but 0.00 acceptance, and the trace model 0.57 fitness but 0.00 acceptance.

Table 5. Generator premise: share of generated traces that exactly match held-out real variants (mean over 15 folds, in %), and the share of the held-out variant set reached (coverage, in %). TLRA (trace-based log representativeness, Table 2) is repeated here: the hit-rate tracks it at Pearson 0.97.

Log	TLRA	ShadowGen	1-gram	Random	Coverage
L1	0.19	1.1	0.4	0.0	4.8
L2	0.80	16.6	10.9	0.01	10.6
L3	0.49	8.6	0.0	0.0	1.4
L4	0.35	0.09	0.0	0.0	0.01
L5	0.95	25.9	5.5	0.0	1.7

(3 shuffles $\times$ 5 folds per log) we train the generator on the train fold only, generate 1,000 traces, and count exact activity-sequence matches of held-out test variants. Since the generator deduplicates against its training variants and the folds share none, a hit cannot be memorization: it is a real future variant produced without ever being seen. Two baselines calibrate it, the 1-gram ablation and uniformly random traces.

Table 5 makes four points. The premise holds: on L5 a quarter of the generated traces are unseen real future variants, while random traces essentially never are (its one nonzero cell is 0.01% on L2's four-activity alphabet). Deep context produces this realism where it matters: on L3 the full model hits 8.6% and the 1-gram never does. The criterion is capped by two structural properties, trace depth and alphabet size, since a hit must reproduce every step over the log's activity set: L4 at 57 events is near zero on depth alone, though its graded MAE 0.016 shows the generator still models it well, and a $k \in \{2, 5, 10\}$ fold sweep confirms depth rather than undertraining ($\leq 0.17\%$ even trained on 90% of L4's variants); the wide-alphabet logs of the 21-log catalog reach the same floor from the alphabet side (Fig. 9). Exact matches are thus a conservative lower bound on

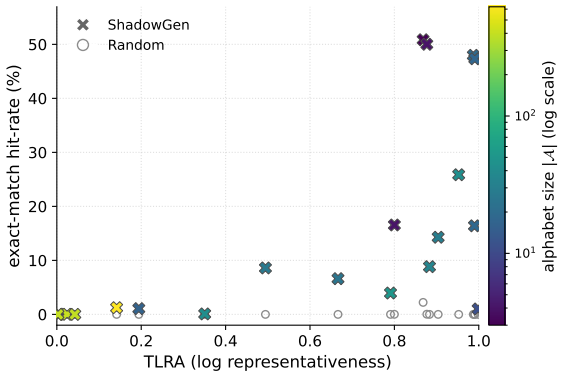

Fig. 9. Generator premise across the 21-log catalog: exact-match hit-rate against log representativeness (TLRA), ShadowGen crosses coloured by alphabet size $|\mathcal{A}|$ vs the uniformly random floor (open circles), one point per log. ShadowGen exceeds random on every log (random never above 2.2%). Two structural properties shape exact-matchability: hit-rate correlates with representativeness (Pearson 0.66 with TLRA) and against alphabet size (Pearson -0.65 on $\log|\mathcal{A}|$). The floor cluster is the large-alphabet municipality logs ($|\mathcal{A}|$ up to 624), which are also the least representative, so both properties push them to zero.

realism, still orders of magnitude above chance. Finally, the rate is a property of the log, not the generator: over the five primary logs it tracks TLRA at Pearson 0.97 (L4 the depth exception). This inverts Sect. 6.2.4, where TLRA was refuted as a *calibration* predictor: representativeness governs how often future behavior can be exact-matched, not how well the metric calibrates [18].

Exact match is a floor. By Levenshtein distance, the shadow log stays close to real behavior even where exact reproduction is out of reach: the share of generated traces within three edits of a held-out variant is 44%/91%/47%/2%/80% on L1–L5, against at most 4.6% for random. The deep-context gap widens here too: on L3 the full model reaches 47% and the 1-gram 0.3%; on the short repetitive L5 the 1-gram edges ahead (90% against 80%). L4 stays low because three edits on a 57-event trace is a 5% tolerance, but its 1-gram and random floors are both zero, so the 2% is real signal.

Across the full 21-log catalog the premise holds broadly (Fig. 9): ShadowGen's exact-match rate meets or exceeds the random floor on every log (strictly on 19 of 21; the two ties are the largest-alphabet municipality logs, where both zero out), and random never exceeds 2.2%. Exact-matchability is shaped by two structural properties of the log, and the figure colours the crosses by the second: the rate rises with representativeness (Pearson 0.66 with TLRA) and falls with alphabet size (Pearson -0.65 on $\log|\mathcal{A}|$, from 3 to 624 activities). This is why the 21-log fit to TLRA alone is looser than the 0.97 within the five primary logs, and it does not weaken the premise: even the hardest logs stay at or above chance.

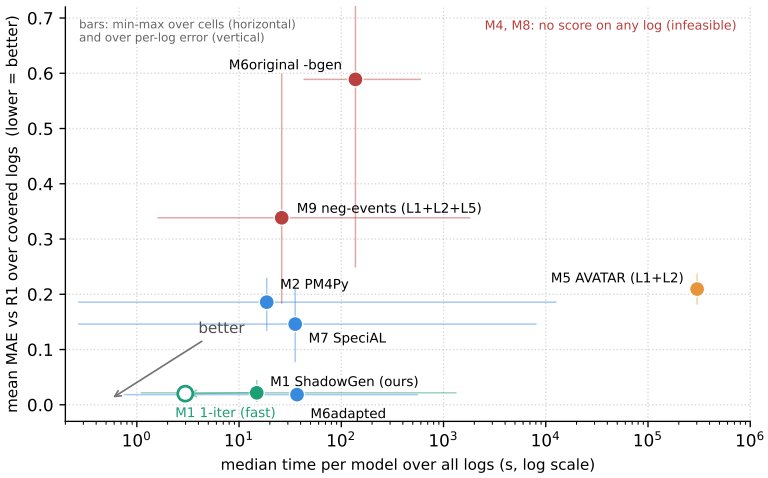

Fig. 10. Speed against accuracy over all five logs: median time per model (log scale) versus mean of the per-log MAEs to R1; lower-left is better. Whiskers show the spread the aggregates hide: horizontal, min to max cell runtime; vertical, min to max per-log MAE. M1 ShadowGen and M6adapted are alone in the accurate corner; M2 PM4Py is fast but never accurate; M9 negative events is fast but the worst-calibrated method that returns (plotted on the three logs it covers); the Entropia `-bgen` F-measure (M6original) is slow and far off; M5 AVATAR is a budget sentinel on all five logs, plotted at its projected CPU cost (∼3–4 days per log, from a measured anchor; Sect. 6.2.1) without a time whisker, its accuracy read from the two small logs it scores off-protocol. Partial-coverage points carry their logs in the label. M4 and M8 do not complete a scorable comparison.

6.2.7 Runtime. At its default single-draw operating point (K=1), ShadowGen scores a model in a median of 3 seconds over all five logs, faster than every published baseline, the next being the trivial PM4Py built-in at 18.6 s, which does not track the ground truth. This is a measured cost, not a projection: timing each cell at K=1 and K=5 on the same machine puts the ratio at 5.0× (median over 24 cells across three logs), the linear scaling expected of K independent draws. The benchmark reports the K=5 mean for its error bars (median 14.9 s per model, worst cell 22 minutes on L4 Inductive-strict, where replay on the discovered net dominates); a single draw reproduces it to within Pearson 0.003 (Sect. 6.4), so K is a validation knob, not an operating cost. Even at K=5 ShadowGen is faster than PM4Py, whose worst cell reaches 3.5 hours against our 22 minutes. The bootstrap adaptation has the same shape at 2.5× the cost (median 36.9 s), and the references are of a different order: R1 reaches a worst cell of 5.7 hours and leave-one-variant-out 46 hours. AVATAR exceeds the budget on every log (Sect. 6.2.1). Figure 10 sets accuracy against time over the whole benchmark. Among methods that agree with the ground truth, ours is the fastest, at K=1 by roughly sixfold over the next.

6.3 Discussion

On five real-life logs spanning 15 thousand to 2.5 million events, the methods that confront a model with resampled or synthesized behavior (M1 ShadowGen, M6adapted) agree with the cross-validation ground truth in both rank and absolute value, whereas methods that inspect structure or aggregate diversity (M2 PM4Py, M7 SpeciAL) do not, on any log. This supports the project's core hypothesis: generalization is a claim about unseen behavior and is therefore best measured by probing models with plausible unseen behavior, instead of analysing the existing structure. That two independently designed generators land on the same calibration strengthens the point: the paradigm carries the result. Our metric reaches this agreement at interactive runtime, at 2.5× lower cost than the bootstrap, with interpretable internals (per-context novelty, mutation flags, the openness profile, the acceptance reading) that neural and bootstrap aggregates do not expose. The honest claim is therefore not "best metric" but "matches the strongest baseline's calibration while probing genuinely unseen behavior that resampling cannot reach, and exposing why a model scored as it did, at comparable cost and with no external toolchain."

6.4 Threats to validity

Seven threats qualify the results; each is stated with what limits it.

Breadth: the study covers five logs and eight discovery configurations per log, but every correlation is computed over six non-degenerate miners (four for SpeciAL on L5 and three for the Entropia `-bgen` tool on L4, as flagged), a deliberately small, interpretable sample (an L1 check over eleven configurations gives the same calibration, Sect. 6.2.2); dataset-specific structure can still favor particular methods, and the AVATAR comparison rests on two off-protocol small-log scorecards read for the construct litmus, so its verdict is correspondingly limited.

Shared measurement core: the metric, R1, R3, and M6adapted all score via token replay, so part of their mutual agreement may reflect a shared fitness notion rather than shared generalization semantics. R1's per-fold rediscovery limits but does not eliminate this, and the acceptance reading was already validated against a separately computed ground truth (R1-accept). We also tested the shared engine directly: recomputing R1 on L1 with cost-zero alignments, a conformance engine independent of token replay, leaves ShadowGen's ranking unchanged (Spearman 1.0, Pearson 0.96 over the five sound nets), so the agreement is not an artifact of the shared engine. Alignment fitness is moreover undefined on unsound nets, since the alignment tool refuses Alpha+, which token replay scores; that is one reason the metric uses token replay.

Ground truth and representativeness: our validation rests on agreement with variant-based hold-out (R1/R2), the strongest available empirical reference because its test traces are provably valid where a shadow trace is only plausible by construction. Both are computed from the log, so both are only as faithful to the system as the log is representative of it, a responsibility of the data-gathering stage rather than of the metric (Sect. 3). The registered prediction

that our calibration error grows as representativeness falls was tested and refuted (Sect. 6.2.4); the bound is real, but TLRA alone does not govern it.

Generator validity: the premise that the shadow log resembles valid future behavior is now tested directly (Sect. 6.2.6): up to 25.9% of generated traces are exact held-out real variants, against at most 0.01% for random traces. The residual threat is that exact matching is a lower-bound criterion: on deep-trace logs it cannot certify realism (L4: 0.09%), where the premise is supported only indirectly by the graded calibration and by the near-match reading (2% of L4 traces within three edits of a held-out variant, both floors zero).

Adapted and incomplete baselines: M6adapted uses token-replay fitness in place of the Entropia precision/recall F-measure, and its sampler is our reimplementation from the paper, validated on L1 (Sect. 5.3); M3 uses a DFG approximation at scale and a proper reachability SDFA where reachability stays finite (24 of 40 cells across the five logs) (Sect. 5.3); M5 has two of 3–5 planned sampling runs on its off-protocol L1/L2 scorecards, and its budget sentinels on all five logs are evidence-based (Sect. 6.2.1); M7's L5 cells use a newer package version than L1–L4 (Sect. 5.3); R2 sampled 50 of 846 variants on L1, and the L4 Inductive-strict cell of R2 is sampled at 50 of 28,457 variants (a full leave-one-variant-out there would need 28,457 rediscoveries).

Calibration provenance: N=6 was selected on one log via a proxy criterion (mutation rate), and the mutated-trace share varies strongly across logs ($\sim 4\%$ on BPI 2017 vs. $\sim 50\%$ on Sepsis), so proposal-distribution effects are dataset-dependent by construction. Relatedly, the variable-order model's advantage over its 1-gram floor is itself log-dependent: on short, simple logs the 1-gram can match or exceed it (Sect. 6.2.4); deep context pays off specifically where decisions depend on it. A sweep over N and τ confirms the setting is not fragile: calibration MAE is a broad plateau for $N \geq 4$ (L1 0.020 to 0.029, L3 0.005 to 0.007; N=6 within noise of the optimum) and barely moves with τ (at most 0.007 across $\tau \in \{2, 5, 10\}$). N=6 is beaten only on the short repetitive L5, where a directly-follows generator already suffices.

Stochastic stability, measured: the generator's own sampling variance is small and carried in every ±std, so a fixed seed hides no fragility. Across the five independent 1,000-trace draws per cell the score std is at most 0.008 (median 0.001), two to three orders of magnitude below the between-model spread it resolves. The metric is in fact single-draw stable: recomputing the four-criteria agreement on each draw separately holds Pearson at or above 0.982 on every log and reproduces the ranking exactly (draw-to-draw std ≤ 0.003 on Pearson, and MAE within 0.007). This is why the shipped default is a single draw (K=1): K=5 is reported for its error bars, not because the result needs averaging.

Replay reproducibility, measured: PM4Py's token replay breaks ties by element iteration order on nets with duplicate labels or silent transitions, and no seed pins it. We measured the drift across independent processes: the trace model moves by up to 0.024, Alpha+ by up to 0.006 (on L4), and every other cell reproduces exactly. The four-criteria agreement is essentially unaffected (the worst-case L4 MAE moves from 0.015 to the tabulated 0.016), and all discovery-

side nondeterminism is pinned (fixed hash seed, models discovered once and cached). The acceptance proxy itself is quantified in Sect. 6.2.5.

7 Conclusion

We presented ShadowGen, a generative N-gram metric for measuring generalization, together with the strict construct definition it implements and a benchmark that validates generalization metrics against variant-based cross-validation under a four-criterion protocol with both construct poles included. The central result: ShadowGen tracks the empirical ground truth on all five real-life logs, reproducing the cross-validation ranking and staying calibrated to it at a median of 3 seconds per model (faster than every published baseline), and it does so while scoring a model artifact in hand, where the cross-validation reference cannot. The benchmark adds findings of independent value. The PM4Py metric is anti-correlated with empirical generalization on four of the five logs and fails the flower litmus on all five. Two published metrics return on too few models to be scored, and the GAN-based AVATAR exceeds the one-hour budget on every log. Rank correlation alone is too weak a validation criterion. An independently designed bootstrap generator lands on the same calibration, corroborating the generative paradigm. The acceptance reading anchors the construct poles and exposes partial-credit inflation, and the generator premise is confirmed directly across the full log catalog.

Open questions remain, and they mark the research opportunities we see. The metric and much of its cross-validation reference share a token-replay core, so part of their agreement reflects a shared conformance notion; an alignment-based acceptance reading would remove the replay proxy and its measured one-sided error, and an alignment-based recomputation of R1 already keeps the ranking (Sect. 6.4). The context bound is calibrated on a single log via a proxy criterion, and per-domain hyperparameter tuning is an open refinement of the fixed-defaults protocol. The strict acceptance reading saturates on deep-trace logs, which invites a graded membership notion for that regime. Several baselines run adapted, degraded, or incomplete; an AVATAR comparison under a GPU budget would let the second generative paradigm be judged at scale. A proper stochastic-automaton conversion of entropic relevance is not among the open items: we implemented it, and it fails the flower litmus (Sect. 6.2.1). The metric itself is feature-frozen; the name ShadowGen reflects its purely generative design.

Declaration on the Use of AI-Based Tools

In preparing this work, we used AI-based assistants (large-language-model and coding tools) for source-code drafting, figure generation, and language editing; all content was reviewed and verified by the authors, who take full responsibility for it.

A Acceptance-reading agreement

Table 6 gives the per-miner acceptance detail on L1 (Gen_{accept} vs the acceptance ground truth R1-accept); Table 7 reports the per-log agreement, the acceptance companion to the graded-fitness scale table (Table 4). Both back Sect. 6.2.5.

Table 6. Acceptance reading on L1 (Gen_{accept}) vs the acceptance ground truth (R1-accept), per miner.

	Trace	Alpha	Alpha+	Heur.	Heur.S	Ind.Inf	Ind.S	Flower
Gen_{accept}	.000	.000	.000	.001	.008	.644	.744	1.00
R1-accept	.000	.000	.000	.028	.043	.783	.996	1.00

Table 7. Acceptance reading (Gen_{accept}) vs the acceptance ground truth (R1-accept): agreement over the six non-degenerate miners per log, plus the two construct poles. Both anchor the trace and flower poles at 0 and 1 on every log (largest deviation 0.003, the L5 trace model; Sect. 6.2.5). *Pearson is uninformative on L4: strict acceptance saturates at trace depth (mean 57 events), leaving only two mid-range cells with signal (Sect. 6.2.5).

Log	Pearson	MAE	Poles (Trace / Flower)
L1 Sepsis	0.998	0.076	0.00 / 1.00
L2 BPI 2013	0.997	0.021	0.00 / 1.00
L3 BPI 2017	0.992	0.039	0.00 / 1.00
L4 BPI 2018	n/a^*	0.057	0.00 / 1.00
L5 BPI 2019	0.999	0.075	0.00 / 1.00

B Raw cross-paradigm scores on L1

Table 8 gives the full per-cell scores behind the L1 calibration (Sect. 6.2.2): the metrics of the L1 comparison and the cross-validation references on each of the seven miners. It is the numeric backing for Figure 7 and Table 3; the R3-Random row is the construct floor made concrete, collapsing in the mid-permissiveness cells (.502, .621) where the real ground truth (.902, .933) does not.

C The bootstrap contamination reading

Table 9 is the numeric backing for the controlled bootstrap study of Sect. 6.2.3: one bred `bsgen` sample on L1, scored four ways against R1. Only the readings free of precision pass the flower litmus.

Table 8. L1 Sepsis: scores per miner, mean ± std where applicable (seven miners of the external comparison). ‡model fitness < 0.5 on the log. †returns too few comparable cells (Sect. 6.2.1). §off-protocol budget sentinel (Sect. 6.2.1): shown for the litmus and Fig. 7, excluded from the agreement statistics.

Method	Alpha‡	Alpha+	Heur.	Heur.S	Ind.Inf	Ind.S	Flower
ShadowGen (ours)	.272±.004	.759±.005	.879±.002	.917±.002	.972±.002	.984±.002	1.00
M2 PM4Py	.913	.919	.830	.900	.880	.902	.913
M5 AVATAR§	.340±.020	.562±.008	.746±.001	.704±.002	.751±.002	.535±.009	.397±.007
M6adapted*	.250±.010	.775±.018	.891±.003	.930±.003	.976±.003	.994±.001	1.00
M7 SpeciAL	.799	.750	1.00	.998	.750	.744	.803
M4 Anti-align.†	*two of seven L1 cells, anti-alignment scale, not comparable*						
M8 Pattern†	*no L1 cell within the hour*						
R1 5-fold CV	.275±.001	.829±.002	.902±.005	.933±.001	.985±.002	1.00	1.00
R2 LOVO	.306±.075	.775±.177	.870±.051	.917±.044	.981±.053	1.00	1.00
R3 Random	.278±.004	.382±.004	.502±.005	.621±.003	.693±.001	.767±.002	1.00

*M6adapted scores the genuine `bsgen` bootstrap sample by token-replay fitness (a graded model–system recall, the construct the bootstrap paper defines [25]), not the Entropia `-bgen` precision/recall F-measure (Sect. 5.3). †M4 returns two L1 cells (Alpha, Heuristics-strict) as anti-alignment scores, not on this table's fitness scale, and exceeds the hour or aborts on the rest; M8 returns no L1 cell within the hour.

Table 9. The same `bsgen` bootstrap sample on L1, scored four ways, vs R1 (six non-degenerate miners). Only the readings free of precision pass the flower litmus.

Reading of the bootstrap sample	Pearson	τ_b	MAE	Flower	litmus
M6original `-bgen` F1(prec, rec)	0.87	0.20	0.59	0.18	fail
its precision component	0.82	0.20	0.68	0.10	fail
its **recall** component	0.98	0.73	0.11	1.00	pass
M6adapted (token-replay)	1.00	1.00	0.018	1.00	pass

References

1. van der Aalst, W.M.P.: Process Mining: Data Science in Action. Springer, Heidelberg, 2nd edn. (2016). https://doi.org/10.1007/978-3-662-49851-4
2. van der Aalst, W.M.P.: Relating process models and event logs — 21 conformance propositions. In: Proc. Int. Workshop on Algorithms & Theories for the Analysis of Event Data (ATAED 2018). CEUR Workshop Proceedings, vol. 2115, pp. 56–74 (2018)
3. van der Aalst, W.M.P., Weijters, A.J.M.M., Maruster, L.: Workflow mining: Discovering process models from event logs. IEEE Transactions on Knowledge and Data Engineering **16**(9), 1128–1142 (2004). https://doi.org/10.1109/TKDE.2004.47
4. Alkhammash, H., Polyvyanyy, A., Moffat, A., García-Bañuelos, L.: Entropic relevance: A mechanism for measuring stochastic process models discovered from event data. Information Systems **107**, 101922 (2022). https://doi.org/10.1016/j.is.2021.101922
5. Augusto, A., Conforti, R., Dumas, M., La Rosa, M., Maggi, F.M., Marrella, A., Mecella, M., Soo, A.: Automated discovery of process models from event logs: Review and benchmark. IEEE Transactions on Knowledge and Data Engineering **31**(4), 686–705 (2019). https://doi.org/10.1109/TKDE.2018.2841877

6. Berti, A., van Zelst, S.J., van der Aalst, W.M.P.: Process mining for Python (PM4Py): Bridging the gap between process- and data science. In: Proceedings of the ICPM Demo Track 2019. CEUR Workshop Proceedings, vol. 2374, pp. 13–16 (2019)
7. Berti, A., van Zelst, S.J., Schuster, D.: PM4Py: A process mining library for Python. Software Impacts **17**, 100556 (2023). https://doi.org/10.1016/j.simpa.2023.100556
8. vanden Broucke, S.K.L.M., Weerdt, J.D., Vanthienen, J., Baesens, B.: Determining process model precision and generalization with weighted artificial negative events. IEEE Transactions on Knowledge and Data Engineering **26**(8), 1877–1889 (2014). https://doi.org/10.1109/TKDE.2013.130
9. Buijs, J.C.A.M., van Dongen, B.F., van der Aalst, W.M.P.: Quality dimensions in process discovery: The importance of fitness, precision, generalization and simplicity. International Journal of Cooperative Information Systems **23**(1), 1440001 (2014). https://doi.org/10.1142/S0218843014400012
10. van Dongen, B.F.: BPI challenge 2017 (2017). https://doi.org/10.4121/uuid:5f3067df-f10b-45da-b98b-86ae4c7a310b
11. van Dongen, B.F.: BPI challenge 2019 (2019). https://doi.org/10.4121/uuid:d06aff4b-79f0-45e6-8ec8-e19730c248f1
12. van Dongen, B.F., Borchert, F.: BPI challenge 2018 (2018). https://doi.org/10.4121/uuid:3301445f-95e8-4ff0-98a4-901f1f204972
13. van Dongen, B.F., Carmona, J., Chatain, T.: A unified approach for measuring precision and generalization based on anti-alignments. In: Business Process Management (BPM 2016). LNCS, vol. 9850, pp. 39–56. Springer (2016). https://doi.org/10.1007/978-3-319-45348-4_3
14. Gale, W.A., Sampson, G.: Good–Turing frequency estimation without tears. Journal of Quantitative Linguistics **2**(3), 217–237 (1995). https://doi.org/10.1080/09296179508590051
15. Janssenswillen, G., Depaire, B.: Towards confirmatory process discovery: Making assertions about the underlying system. Business & Information Systems Engineering **61**(6), 713–728 (2019). https://doi.org/10.1007/s12599-018-0567-8
16. Janssenswillen, G., Donders, N., Jouck, T., Depaire, B.: A comparative study of existing quality measures for process discovery. Information Systems **71**, 1–15 (2017). https://doi.org/10.1016/j.is.2017.06.002
17. Kabierski, M., Richter, M., Weidlich, M.: Addressing the log representativeness problem using species discovery. In: Proceedings of the 5th International Conference on Process Mining (ICPM 2023). pp. 65–72. IEEE (2023). https://doi.org/10.1109/ICPM60904.2023.10271952
18. Karunaratne, A., Polyvyanyy, A., Moffat, A.: Generalization estimation in process mining: the impact of event data quality. Process Science **2**(1), 20 (2025). https://doi.org/10.1007/s44311-025-00027-3, extended version of the ICPM 2024 paper on log representativeness
19. Katz, S.M.: Estimation of probabilities from sparse data for the language model component of a speech recognizer. IEEE Transactions on Acoustics, Speech, and Signal Processing **35**(3), 400–401 (1987). https://doi.org/10.1109/TASSP.1987.1165125
20. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Discovering block-structured process models from event logs — A constructive approach. In: Application and Theory of Petri Nets and Concurrency (PETRI NETS 2013). LNCS, vol. 7927, pp. 311–329. Springer (2013). https://doi.org/10.1007/978-3-642-38697-8_17

21. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Discovering block-structured process models from event logs containing infrequent behaviour. In: Business Process Management Workshops (BPM 2013). LNBIP, vol. 171, pp. 66–78. Springer (2014). https://doi.org/10.1007/978-3-319-06257-0_6
22. Leemans, S.J.J., Syring, A.F., van der Aalst, W.M.P.: Earth movers' stochastic conformance checking. In: Business Process Management Forum (BPM Forum 2019). LNBIP, vol. 360, pp. 127–143. Springer (2019). https://doi.org/10.1007/978-3-030-26643-1_8
23. Mannhardt, F.: Sepsis cases — event log (2016). https://doi.org/10.4121/uuid:915d2bfb-7e84-49ad-a286-dc35f063a460
24. Polyvyanyy, A., Alkhammash, H., Di Ciccio, C., García-Bañuelos, L., Kalenkova, A.A., Leemans, S.J.J., Mendling, J., Moffat, A., Weidlich, M.: Entropia: A family of entropy-based conformance checking measures for process mining. arXiv:2008.09558 (2020). https://doi.org/10.48550/arXiv.2008.09558
25. Polyvyanyy, A., Moffat, A., García-Bañuelos, L.: Bootstrapping generalization of process models discovered from event data. In: Advanced Information Systems Engineering (CAiSE 2022). LNCS, vol. 13295, pp. 36–54. Springer (2022). https://doi.org/10.1007/978-3-031-07472-1_3
26. Reißner, D., Armas-Cervantes, A., La Rosa, M.: Generalization in automated process discovery: A framework based on event log patterns. arXiv:2203.14079 (2022). https://doi.org/10.48550/arXiv.2203.14079
27. Rozinat, A., van der Aalst, W.M.P.: Conformance checking of processes based on monitoring real behavior. Information Systems **33**(1), 64–95 (2008). https://doi.org/10.1016/j.is.2007.07.001
28. Steeman, W.: BPI challenge 2013, incidents (2013). https://doi.org/10.4121/uuid:500573e6-accc-4b0c-9576-aa5468b10cee
29. Syring, A.F., Tax, N., van der Aalst, W.M.P.: Evaluating conformance measures in process mining using conformance propositions. In: Transactions on Petri Nets and Other Models of Concurrency XIV, LNCS, vol. 11790, pp. 192–221. Springer (2019). https://doi.org/10.1007/978-3-662-60651-3_8
30. Theis, J., Darabi, H.: Adversarial system variant approximation to quantify process model generalization. IEEE Access **8**, 194410–194427 (2020). https://doi.org/10.1109/ACCESS.2020.3033450
31. Weijters, A.J.M.M., van der Aalst, W.M.P., de Medeiros, A.K.A.: Process mining with the HeuristicsMiner algorithm. Tech. Rep. WP 166, Eindhoven University of Technology (2006)